

# Práctica: tu primer aporte a un repositorio

En este repositorio hay una página web que se arma entre todos. Tu trabajo es agregar tu propia tarjeta al muro, subir el cambio a GitHub y levantar la página con Docker para verla funcionando en tu navegador.

Carrera de Ingeniería Comercial | UAJMS, Sede Yacuiba  
Duración estimada: 60 minutos | Modalidad: individual

## Antes de empezar

Verificá que tenés las dos herramientas instaladas. Si alguno de los dos comandos falla, avisá antes de seguir:

```
git --version
docker --version
```

Y que Git sabe quién sos. Esto define el nombre que va a quedar firmando tus commits:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"
```

## Parte 1 — Aportar con Git

### 1. Traé el repositorio a tu máquina

```
git clone <URL-del-repositorio>
cd practica-git-docker
```

El clone se hace una sola vez. A partir de ahí trabajás dentro de esa carpeta.

### 2. Creá tu rama

Nunca trabajes directo sobre main. Usá tu apellido para nombrarla:

```
git checkout -b aporte/gutierrez
```

Confirmá en qué rama estás parado con `git branch`: la tuya tiene un asterisco.

### 3. Modificá index.html

Abrí el archivo y buscá el bloque comentado que dice **COPIÁ EL BLOQUE DE ABAJO**. Copiá esa tarjeta, pegala **antes** del comentario FIN DE LOS APORTES y cambiá cuatro cosas:

- tu nombre y apellido;
- tu carrera y semestre;
- el mensaje: una o dos líneas tuyas, por qué te interesa el curso o qué querés aprender;
- tu usuario de GitHub.

No borres ni edites las tarjetas de tus compañeros. Tu cambio tiene que sumar, no reemplazar.

### 4. Revisá qué cambiaste antes de guardar nada

```
git status
git diff
```

Leé el diff: tiene que haber solo líneas agregadas (con `+`) y ninguna borrada (con `-`). Si ves líneas borradas que no son tuyas, corregí antes de continuar.

### 5. Guardá el cambio en un commit

```
git add index.html
git commit -m "Agrega aporte de Ana Gutiérrez al muro"
```

El mensaje va en presente y describe qué hace el cambio. No sirven "cambios", "prueba" ni "asdasd": dentro de seis meses ese mensaje es lo único que va a quedar para entender qué pasó.

### 6. Subí tu rama y abrí un Pull Request

```
git push -u origin aporte/gutierrez
```

Entrá al repositorio en GitHub, abrí el Pull Request hacia main y escribí una línea explicando tu aporte. El `-u` solo hace falta la primera vez que subís esa rama; después alcanza con `git push`.

## Parte 2 — Levantar la página con Docker

Desde la carpeta del proyecto, donde está el Dockerfile:

```
docker build -t muro-aportes .
docker run -d -p 8080:80 --name muro muro-aportes
```

Abri http://localhost:8080 en el navegador. Deberías ver el muro con tu tarjeta. El -p 8080:80 conecta el puerto 8080 de tu computadora con el 80 del contenedor, que es donde escucha nginx.

Si cambiás el HTML o el CSS, la página del contenedor no se actualiza sola. Tenés que detener el contenedor, eliminarlo, volver a construir la imagen y correrla de nuevo. Ese es justamente el punto: la imagen es una foto del proyecto en un momento dado.

```
docker stop muro
docker rm muro
docker build -t muro-aportes .
docker run -d -p 8080:80 --name muro muro-aportes
```

## Parte 3 (opcional) — Provocá un conflicto a propósito

En parejas, los dos modifican **la misma línea** del pie de página en ramas distintas y después hacen el merge. Git va a marcar el conflicto así dentro del archivo:

```
<<<<<< HEAD
    texto de tu rama
=====
    texto de la otra rama
>>>>>> aporte/villa
```

Resolverlo es decidir a mano con qué texto se queda el archivo y borrar las tres líneas marcadoras. Después:

```
git add index.html
git commit -m "Resuelve conflicto en el pie de página"
```

## Entrega y evaluación

- Rama subida al repositorio remoto.
- Pull Request abierto, con descripción.
- Captura de pantalla de la página corriendo en localhost:8080.

| Criterio                                             | Puntos |
|------------------------------------------------------|--------|
| Rama creada con el nombre indicado                   | 1      |
| La tarjeta se agrega sin borrar aportes ajenos       | 2      |
| Mensaje de commit claro y descriptivo                | 2      |
| Pull Request abierto y explicado                     | 2      |
| Imagen construida y contenedor funcionando (captura) | 3      |
| Total                                                | 10     |

# Cheatsheet de comandos — Git

| Comando                                | Qué hace                                            |
|----------------------------------------|-----------------------------------------------------|
| Configuración inicial                  |                                                     |
| git config --global user.name "Nombre" | Define el nombre que firma tus commits              |
| git config --global user.email "mail"  | Define tu correo en los commits                     |
| git config --list                      | Muestra la configuración actual                     |
| Empezar un repositorio                 |                                                     |
| git init                               | Convierte una carpeta en repositorio Git            |
| git clone <url>                        | Descarga un repositorio remoto completo             |
| Ver el estado y los cambios            |                                                     |
| git status                             | Qué archivos cambiaron y cuáles están preparados    |
| git diff                               | Muestra los cambios todavía no preparados           |
| git diff --staged                      | Muestra lo que ya está preparado para el commit     |
| git log --oneline --graph              | Historial resumido, una línea por commit            |
| git show <hash>                        | Ve el contenido de un commit puntual                |
| Guardar cambios                        |                                                     |
| git add <archivo>                      | Prepara un archivo para el próximo commit           |
| git add .                              | Prepara todos los cambios de la carpeta             |
| git commit -m "mensaje"                | Guarda los cambios preparados en el historial       |
| git commit --amend                     | Corrige el mensaje o el contenido del último commit |
| git restore <archivo>                  | Descarta los cambios no guardados de un archivo     |
| git restore --staged <archivo>         | Saca un archivo del área de preparación             |
| Ramas                                  |                                                     |
| git branch                             | Lista las ramas; la actual lleva asterisco          |
| git checkout -b <rama>                 | Crea una rama nueva y se cambia a ella              |
| git checkout <rama>                    | Se cambia a una rama existente                      |
| git merge <rama>                       | Trae los cambios de otra rama a la actual           |
| git branch -d <rama>                   | Borra una rama ya fusionada                         |
| Trabajar con el remoto                 |                                                     |
| git remote -v                          | Muestra a qué repositorio remoto apunta             |
| git push -u origin <rama>              | Sube tu rama por primera vez                        |
| git push                               | Sube los commits nuevos de la rama actual           |
| git pull                               | Trae y aplica los cambios del remoto                |
| git fetch                              | Trae los cambios sin aplicarlos todavía             |

| Comando                                                      | Qué hace                                                  |
|--------------------------------------------------------------|-----------------------------------------------------------|
| Imágenes                                                     |                                                           |
| <code>docker build -t &lt;nombre&gt; .</code>                | Construye una imagen con el Dockerfile de la carpeta      |
| <code>docker images</code>                                   | Lista las imágenes que tenés en la máquina                |
| <code>docker pull &lt;imagen&gt;</code>                      | Descarga una imagen desde Docker Hub                      |
| <code>docker rmi &lt;imagen&gt;</code>                       | Elimina una imagen                                        |
| Contenedores                                                 |                                                           |
| <code>docker run -d -p 8080:80 &lt;imagen&gt;</code>         | Corre un contenedor en segundo plano publicando el puerto |
| <code>docker run --name &lt;nombre&gt; &lt;imagen&gt;</code> | Corre el contenedor con un nombre propio                  |
| <code>docker ps</code>                                       | Lista los contenedores que están corriendo                |
| <code>docker ps -a</code>                                    | Lista todos, incluso los detenidos                        |
| <code>docker stop &lt;nombre&gt;</code>                      | Detiene un contenedor                                     |
| <code>docker start &lt;nombre&gt;</code>                     | Vuelve a levantar uno detenido                            |
| <code>docker rm &lt;nombre&gt;</code>                        | Elimina un contenedor detenido                            |
| Inspeccionar y depurar                                       |                                                           |
| <code>docker logs &lt;nombre&gt;</code>                      | Muestra la salida del contenedor                          |
| <code>docker logs -f &lt;nombre&gt;</code>                   | Sigue los logs en vivo                                    |
| <code>docker exec -it &lt;nombre&gt; sh</code>               | Abre una terminal dentro del contenedor                   |
| <code>docker inspect &lt;nombre&gt;</code>                   | Muestra la configuración completa en detalle              |
| Limpieza                                                     |                                                           |
| <code>docker stop \$(docker ps -q)</code>                    | Detiene todos los contenedores activos                    |
| <code>docker system df</code>                                | Muestra cuánto espacio está ocupando Docker               |
| <code>docker system prune</code>                             | Borra contenedores, redes e imágenes sin usar             |
| Instrucciones del Dockerfile usadas en esta práctica         |                                                           |
| <code>FROM nginx:alpine</code>                               | Imagen base sobre la que se construye                     |
| <code>COPY index.html /usr/share/nginx/html/</code>          | Copia un archivo tuyo dentro de la imagen                 |
| <code>EXPOSE 80</code>                                       | Declara el puerto en el que escucha el servicio           |

Si algo sale mal

| Síntoma                                     | Qué probar                                                                             |
|---------------------------------------------|----------------------------------------------------------------------------------------|
| «fatal: not a git repository»               | No estás dentro de la carpeta del proyecto. Usá <code>cd</code> para entrar.           |
| El push pide usuario y contraseña y falla   | GitHub ya no acepta contraseña: usá un token personal o configurá SSH.                 |
| «rejected — non-fast-forward» al hacer push | Hay commits nuevos en el remoto. Hacé <code>git pull</code> , resolvé y volvé a subir. |
| «port is already allocated»                 | El 8080 está ocupado. Probá con <code>-p 8081:80</code> .                              |
| «name is already in use by container»       | Ya existe un contenedor con ese nombre: <code>docker rm muro</code> .                  |
| Cambié el HTML y la página sigue igual      | Falta reconstruir: <code>docker build</code> otra vez y correr un contenedor nuevo.    |
| «Cannot connect to the Docker daemon»       | Docker Desktop no está abierto o el servicio no está corriendo.                        |